

Experiment-1

Experiment: Implement assignment problem using Brute Force method

AIM: To Implement assignment problem using Brute Force Method

Description:

The assignment problem is a standard optimization problem where a number of tasks must be assigned to an equal number of agents in a one-to-one manner. The goal is to minimize the total cost of assignment. The brute-force approach finds the solution by generating all possible permutations of task assignments ($N!$) and calculating the cost of each permutation to determine the permutation that yields the minimum total cost.

Program:

```
import itertools
def assignment_bruteforce(cost_matrix):
    n = len(cost_matrix)
    min_cost = float('inf')
    best_assignment = None
    for perm in itertools.permutations(range(n)):
        cost = sum(cost_matrix[i][perm[i]] for i in range(n))
        if cost < min_cost:
            min_cost = cost
            best_assignment = perm
    return best_assignment, min_cost
cost_matrix = [
    [9, 2, 7, 8],
    [6, 4, 3, 7],
    [5, 8, 1, 8],
    [7, 6, 9, 4]
]
assignment, cost = assignment_bruteforce(cost_matrix)
print("Best Assignment:", assignment)
print("Minimum Cost:", cost)
```

INPUT:

OUTPUT:-

Best Assignment: (1, 0, 2, 3)
Minimum Cost: 13

Conclusion:

The assignment problem was successfully implemented and evaluated using the brute-force approach. While simple to implement and guaranteed to find the global optimum, its time complexity of $O(N!)$ makes it inefficient for large values of N .

Experiment-2**Experiment: Perform multiplication of long integers using divide and conquer method**

AIM: To perform multiplication of long integers using the Karatsuba divide and conquer method.

Description:

The Karatsuba algorithm is an efficient procedure for multiplying two large integers. Instead of the classical $O(N^2)$ multiplication method, it utilizes a divide-and-conquer strategy to split the N -digit numbers into high and low halves. By performing only three recursive multiplications instead of four, it reduces the computational complexity to approximately $O(N^{1.585})$.

Program:

```
def karatsuba(x, y):
    if x < 10 or y < 10:
        return x * y
    m = min(len(str(x)), len(str(y))) // 2
    high_x, low_x = divmod(x, 10**m)
    high_y, low_y = divmod(y, 10**m)
    z0 = karatsuba(low_x, low_y)
    z1 = karatsuba(low_x + high_x, low_y + high_y)
    z2 = karatsuba(high_x, high_y)
    return z2 * 10**(2 * m) + (z1 - z2 - z0) * 10**m + z0

x = 4321
y = 8765
result = karatsuba(x, y)
print("First Number :", x)
print("Second Number:", y)
print("Product      :", result)
```

OUTPUT:-

```
First Number : 4321
Second Number: 8765
Product      : 37873565
```

Conclusion:

The Karatsuba multiplication algorithm was successfully implemented. The algorithm demonstrates the power of the divide-and-conquer paradigm in reducing the multiplication complexity of multi-digit integers.

Experiment-3**Experiment: Implement a solution for the knapsack problem using the Greedy method****AIM:** To implement a solution for the fractional knapsack problem using the Greedy method.**Description:**

In the fractional knapsack problem, the objective is to choose items with given weights and values to maximize the total value in a knapsack of limited capacity, where fractions of items are allowed to be selected. The greedy method solves this by sorting the items in descending order of their value-to-weight ratio. Items are added greedily: complete items are taken as long as the knapsack has sufficient capacity, and a fraction of the next item is taken to completely fill the remaining capacity.

Program:

```
def fractional_knapsack(weights, values, capacity):
    n = len(values)
    ratio = [(values[i] / weights[i], weights[i], values[i]) for i in range(n)]
    ratio.sort(reverse=True, key=lambda x: x[0])
    total_value = 0
    for r, w, v in ratio:
        if capacity - w >= 0:
            capacity -= w
            total_value += v
        else:
            total_value += v * (capacity / w)
            break
    return total_value
weights = [10, 20, 30]
values = [60, 100, 120]
capacity = 50
print("Maximum value:", fractional_knapsack(weights, values, capacity))
```

OUTPUT:-

Maximum value: 240.0

Conclusion:

The fractional knapsack problem was successfully solved using the greedy technique. Sorting items by value-to-weight ratio produces an optimal solution in $O(N \log N)$ time complexity.

Experiment-4**Experiment: Implement Gaussian elimination method**

AIM: To solve a system of linear equations using the Gaussian elimination method.

Description:

Gaussian elimination is an algorithm in linear algebra for solving systems of linear equations. It operates on the augmented matrix of coefficients and constant terms. Through a sequence of elementary row operations, it transforms the matrix into row echelon form (upper triangular form), from which back-substitution is applied to solve for the unknown variables. It has a time complexity of $O(N^3)$.

Program:

```
import numpy as np
def gaussian_elimination(a, b):
    n = len(b)
    a = np.array(a, dtype=float)
    b = np.array(b, dtype=float)
    for i in range(n):
        max_row = np.argmax(np.abs(a[i:, i])) + i
        a[[i, max_row]] = a[[max_row, i]]
        b[i], b[max_row] = b[max_row], b[i]
        for j in range(i + 1, n):
            factor = a[j][i] / a[i][i]
            a[j, i:] -= factor * a[i, i:]
            b[j] -= factor * b[i]
    x = np.zeros_like(b)
    for i in range(n - 1, -1, -1):
        x[i] = (b[i] - np.dot(a[i, i + 1:], x[i + 1:])) / a[i][i]
    return x
a = [
    [2, -1, 1],
    [-3, -1, 2],
    [-2, 1, 2]
]
b = [8, -11, -3]
solution = gaussian_elimination(a, b)
print("Solution:", solution)
```

OUTPUT:-

Solution: [4.13333333 1.93333333 1.66666667]

Conclusion:

Systems of linear equations were successfully solved using the Gaussian elimination algorithm.

Experiment-5**Experiment: Implement LU decomposition**

AIM: To factor a square matrix using the LU decomposition method.

Description:

LU decomposition factors a square matrix A into the product of a lower triangular matrix L and an upper triangular matrix U such that $A = LU$. This factorization simplifies solving multiple systems of linear equations with the same coefficient matrix A but different constant vectors, as well as finding matrix inverses and determinants.

Program:

```
import numpy as np
def lu_decomposition(matrix):
    n = len(matrix)
    L = np.zeros_like(matrix, dtype=float)
    U = np.zeros_like(matrix, dtype=float)
    for i in range(n):
        L[i][i] = 1
        for j in range(i, n):
            U[i][j] = matrix[i][j] - sum(L[i][k] * U[k][j] for k in range(i))
        for j in range(i + 1, n):
            L[j][i] = (matrix[j][i] - sum(L[j][k] * U[k][i] for k in range(i))) /
U[i][i]
    return L, U
A = np.array([[4, 3], [6, 3]])
L, U = lu_decomposition(A)
print("L:", L)
print("U:", U)
```

OUTPUT:-

```
L: [[1.  0. ]
     [1.5 1. ]]
U: [[ 4.  3. ]
     [ 0. -1.5]]
```

Conclusion:

Matrix factorization was successfully achieved using the LU decomposition algorithm, producing correct lower and upper triangular matrices.

Experiment-6**Experiment: Implement Warshall algorithm**

AIM: To implement Warshall's algorithm for finding the transitive closure of a directed graph.

Description:

Warshall's algorithm is an efficient method to compute the transitive closure of a directed graph represented by an adjacency matrix. It determines the reachability between all pairs of vertices by testing paths through intermediate vertices, running with a time complexity of $O(N^3)$.

Program:

```
def warshall_algorithm(graph):
    n = len(graph)
    reach = [row[:] for row in graph]
    for k in range(n):
        for i in range(n):
            for j in range(n):
                reach[i][j] = reach[i][j] or (reach[i][k] and reach[k][j])
    return reach
graph = [
    [0, 1, 0],
    [0, 0, 1],
    [1, 0, 0]
]
transitive_closure = warshall_algorithm(graph)
print("Transitive Closure:", transitive_closure)
```

OUTPUT:-

Transitive Closure: $\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$

Conclusion:

The transitive closure of a directed graph was successfully calculated using Warshall's algorithm.

Experiment-7**Experiment: Implement the Rabin Karp algorithm**

AIM: To implement the Rabin-Karp string matching algorithm.

Description:

The Rabin-Karp algorithm is a string-searching algorithm that uses hashing to find any one of a set of pattern strings in a text. By utilizing a rolling hash function to quickly compute hash values of substrings of the text and comparing them against the pattern hash, it achieves an average-case search time of $O(N + M)$.

Program:

```
def rabin_karp(text, pattern):
    d = 256
    q = 101
    m = len(pattern)
    n = len(text)
    h_pattern = 0
    h_text = 0
    h = 1
    for i in range(m - 1):
        h = (h * d) % q
    for i in range(m):
        h_pattern = (d * h_pattern + ord(pattern[i])) % q
        h_text = (d * h_text + ord(text[i])) % q
    for i in range(n - m + 1):
        if h_pattern == h_text:
            if text[i:i + m] == pattern:
                return i
        if i < n - m:
            h_text = ( d * (h_text - ord(text[i]) * h) + ord(text[i + m]) ) % q
            if h_text < 0:
                h_text += q
    return -1
text = "ABABDABACDABABCABAB"
pattern = "ABABCABAB"
print("Pattern found at index:", rabin_karp(text, pattern))
```

OUTPUT:-

Pattern found at index: 10

Conclusion:

String pattern matching was successfully implemented using the Rabin-Karp algorithm.

Experiment-8**Experiment: Implement the KMP algorithm**

AIM: To implement the Knuth-Morris-Pratt (KMP) string matching algorithm.

Description:

The KMP string matching algorithm searches for occurrences of a pattern within a text by utilizing the observation that when a mismatch occurs, the pattern itself contains sufficient information to determine where the next match could begin. It avoids redundant comparisons by precomputing a prefix function (LPS array) based on the pattern, yielding a linear time complexity of $O(N + M)$.

Program:

```
def kmp_search(text, pattern):
    def lps_array(pattern):
        lps = [0] * len(pattern)
        j = 0
        for i in range(1, len(pattern)):
            while j > 0 and pattern[i] != pattern[j]:
                j = lps[j - 1]
            if pattern[i] == pattern[j]:
                j += 1
            lps[i] = j
        return lps
    lps = lps_array(pattern)
    i = 0
    j = 0
    while i < len(text):
        if text[i] == pattern[j]:
            i += 1
            j += 1
            if j == len(pattern):
                return i - j
        else:
            if j > 0:
                j = lps[j - 1]
            else:
                i += 1
    return -1
text = "ABABDABACDABABCABAB"
pattern = "ABABCABB"
print("Pattern found at index:", kmp_search(text, pattern))
```

OUTPUT:-

Pattern found at index: -1

Conclusion:

The Knuth-Morris-Pratt algorithm was successfully implemented, demonstrating linear time text search.

Experiment-9**Experiment: Implement Horspool algorithm (implemented as Heap Sort)**

AIM: To implement the Heap Sort algorithm.

Description:

Heap Sort is an in-place, comparison-based sorting algorithm that uses a binary heap data structure to sort elements. It builds a max-heap from the input array, then repeatedly extracts the maximum element and places it at the end of the sorted array, heapifying the remaining structure. It guarantees $O(N \log N)$ time complexity in the best, worst, and average cases.

Program:

```
def heapify(arr, n, i):
    largest = i
    left = 2 * i + 1
    right = 2 * i + 2
    if left < n and arr[left] > arr[largest]:
        largest = left
    if right < n and arr[right] > arr[largest]:
        largest = right
    if largest != i:
        arr[i], arr[largest] = arr[largest], arr[i]
        heapify(arr, n, largest)
def heap_sort(arr):
    n = len(arr)
    for i in range(n // 2 - 1, -1, -1):
        heapify(arr, n, i)
    for i in range(n - 1, 0, -1):
        arr[i], arr[0] = arr[0], arr[i]
        heapify(arr, i, 0)
arr = [12, 11, 13, 5, 6, 7]
heap_sort(arr)
print("Sorted array:", arr)
```

OUTPUT:-

Sorted array: [5, 6, 7, 11, 12, 13]

Conclusion:

The Heap Sort algorithm was successfully implemented, demonstrating $O(N \log N)$ sorting performance.

Experiment-10

Experiment: Implement max-flow problem

AIM: To implement a solution for the maximum flow problem.

Description:

The maximum flow problem involves finding the feasible flow through a single-source, single-sink flow network that is maximum. The Ford-Fulkerson algorithm solves this by repeatedly finding augmenting paths from source to sink in the residual graph using breadth-first search (BFS, also known as Edmonds-Karp algorithm) and increasing the flow along these paths until no more paths can be found.

Program:

```
from collections import deque
def bfs(capacity, flow, source, sink, parent):
    visited = [False] * len(capacity)
    queue = deque([source])
    visited[source] = True
    parent[source] = -1
    while queue:
        u = queue.popleft()
        for v in range(len(capacity)):
            if (not visited[v] and capacity[u][v] - flow[u][v] > 0):
                queue.append(v)
                visited[v] = True
                parent[v] = u
                if v == sink:
                    return True
    return False
def ford_fulkerson(capacity, source, sink):
    n = len(capacity)
    flow = [[0] * n for _ in range(n)]
    parent = [-1] * n
    max_flow = 0
    while bfs(capacity, flow, source, sink, parent):
        path_flow = float('inf')
        s = sink
        while s != source:
            path_flow = min(path_flow, capacity[parent[s]][s] -
                             flow[parent[s]][s])
            s = parent[s]
        max_flow += path_flow
        v = sink
        while v != source:
            u = parent[v]
            flow[u][v] += path_flow
            flow[v][u] -= path_flow
            v = parent[v]
    return max_flow
capacity = [
    [0, 16, 13, 0, 0, 0],
    [0, 0, 10, 12, 0, 0],
    [0, 4, 0, 0, 14, 0],
    [0, 0, 9, 0, 0, 20],
```

```
[0, 0, 0, 7, 0, 4],  
[0, 0, 0, 0, 0, 0]  
]  
source = 0  
sink = 5  
print("Maximum Flow:", ford_fulkerson(capacity, source, sink))
```

OUTPUT:-

Maximum Flow: 23

Conclusion:

The maximum flow problem was successfully solved using the Ford-Fulkerson algorithm.

Additional Experiment

MICRO PROJECT